



Department of Informatics

"Distributed Data Processing with Apache Spark."

Distributed Computing Project

MSc Data & Web Science

Course: Distributed Data Processing - 2025-2026

Contributors:

Evangelia Grousi (AM: 229)

Georgios Varatis (AM: 219)

Github Repository:

<https://github.com/VaratisG/DistributedComputing>

Table of Contents

1. Introduction.....	3
2. Architecture and Design Patterns	3
2.1 The Data Factory Pattern (config.py)	3
2.2 Project Structure	3
2.3 Unified Benchmark Runner (src/benchmark.py)	4
3. Part A - All-Pairs Matching Problem.....	4
3.1 Naive MapReduce (naive.py)	4
3.2 Group-Based MapReduce (group.py).....	5
3.3 Spark SQL with Catalyst Optimizer (sql.py)	5
3.4 Bonus - Afrati-Ullman / Fano Plane (partA_bonus.py).....	5
4. Performance Evaluation - Part A.....	6
4.1 Results	6
4.2 Analysis.....	6
5. Part B - Three-Way Join A(x,y) Join B(y,z) Join C(z,w)	6
5.1 Direct Ternary Join (ternary.py)	7
5.2 Two Successive Binary Joins (binary.py)	7
5.3 Spark SQL - Catalyst Optimizer (join_sql.py)	7
5.4 Performance Evaluation - Part B.....	7
5.5 Analysis.....	8
6. Conclusions.....	8
7. Execution Instructions.....	9

1. Introduction

This report documents the design decisions, implementation details, and empirical evaluation of a distributed data-processing project built on Apache Spark (PySpark). The project is divided into two parts:

- **Part A - All-Pairs Matching:** given a collection of N elements, enumerate every unique unordered pair (u, v) where $u \neq v$. Three distinct MapReduce strategies are implemented and benchmarked, plus a theoretically optimal bonus implementation.
- **Part B - Three-Way Relational Join:** compute $A(x,y) \text{ Join } B(y,z) \text{ Join } C(z,w)$ via a direct ternary join, two successive binary joins, and a Spark SQL query.

All code is written in Python and executed on a local single-node Spark instance. The dataset used is `person.csv`, a real-world CSV file from which up to 1 million rows are loaded depending on the benchmark configuration (N).

2. Architecture and Design Patterns

2.1 The Data Factory Pattern (`config.py`)

A fundamental design goal was to keep every MapReduce script fully decoupled from data-source concerns. This was achieved by centralising all configuration and data loading into a single module, `config.py`, which acts as a factory.

- **Decoupled Logic:** the algorithmic scripts (`naive.py`, `group.py`, `sql.py`) contain only the **MapReduce** logic. They call `config.get_part_a_dataset(spark, sc)` and receive an RDD and an element count, this way they remain entirely blind to whether the data is synthetic or real.
- **Feature Flagging via Test Mode:** setting `TEST_MODE = True` switches the loader to a synthetic RDD of `MOCK_N` integers. This allows rapid algorithm validation without touching any Spark I/O. Setting `TEST_MODE = False` loads the real CSV and applies `LIMIT_N` to control dataset size.
- **Sequential Indexing with `zipWithIndex()`:** real CSV rows contain arbitrary string IDs. To make modular arithmetic meaningful for the group-based approach, the loader uses `zipWithIndex()` to map every record to a contiguous $0 \dots N-1$ integer sequence.

```
# config.py - data factory
def get_part_a_dataset(spark, sc):
    if TEST_MODE:
        rdd = sc.parallelize(range(MOCK_N))
        return rdd, MOCK_N
    else:
        df = spark.read.csv(DATASET_PATH, header=True, inferSchema=True)
        rdd = df.limit(LIMIT_N).rdd.zipWithIndex().map(lambda x: x[1])
        return rdd, LIMIT_N
```

2.2 Project Structure

```
Spark_Project/
├── config.py           # Centralised config & data-loader factory
├── benchmark_output.txt # The results from the benchmark.py tests
```

```

├── Dataset/
│   └── person.csv          # Source dataset
└── src/
    ├── benchmark.py        # Full benchmark matrix
    ├── partA/
    │   ├── naive.py        # Part A - Naive MapReduce
    │   ├── group.py        # Part A - Group-Based MapReduce
    │   ├── sql.py          # Part A - Spark SQL / Catalyst
    │   └── partA_bonus.py   # Part A Bonus - Afrati-Ullman / Fano Plane
    └── partB/
        ├── ternary.py       # Part B - Direct Ternary Join (Hypercube)
        ├── binary.py        # Part B - Two Binary Joins
        └── join_sql.py       # Part B - Spark SQL

```

2.3 Unified Benchmark Runner (src/benchmark.py)

All benchmark results in this report were produced by a single script, **src/benchmark.py**, which runs every **Part A** and **Part B** approach across the entire test matrix (*BENCH_A_SIZES*, *BENCH_B_SELECTIVITY*, *BENCH_B_SCALE* from *config.py*) inside one Spark session. Reusing a single warm JVM across all experiments avoids paying the ~15-second Spark startup cost six separate times and keeps the six approaches directly comparable.

Benchmark.py was not written by hand like the other .py files in the project. An LLM made it from scratch after the individual scripts were finished, and its purpose was to control the entire test matrix from one entry point. The natural alternative (importing naive.py, group.py, ternary.py, etc. and calling a function from each) is not possible because each of those standalone scripts creates its own **SparkSession** when the module is loaded. This would create multiple sessions and defeat the whole point of the unified runner. Copying the six algorithms that are already in benchmark.py was the most realistic option.

Benchmark output is written simultaneously to the console and to benchmark_output.txt (overwritten on each run) via a small log() helper, so the raw numbers can be inspected later without having to re-run the whole matrix.

3. Part A - All-Pairs Matching Problem

The All-Pairs Matching problem includes the comprehensive enumeration of every distinct, unordered pair (u,v) from a set S comprising N elements, **resulting in a total of $N * \frac{(N-1)}{2}$** pairs. To solve this, we used three MapReduce methods, each of which had a different trade-off between the amount of communication needed and the way the load was spread out among reducers.

3.1 Naive MapReduce (naive.py)

The Naïve approach gives each unique pair (i, j) its own reducer. The mapper sends a message to every reducer that is in charge of a pair (min(i, j), max(i, j)) for all j that are not equal to i. Because of this, each reducer gets exactly two elements to check the match.

- **Replication Rate:** Each element is copied $N - 1$ times, so there are $O(N)$ messages for each element.
- **Total Network Traffic:** The shuffle phase sends out $N * (N - 1)$ messages, which makes communication complexity $O(N^2)$.
- **Reducer Count:** The job is split up among $N * \frac{(N-1)}{2}$ reducers.

This method is not effective at scale because the communication cost goes up by a factor of two. The shuffle will made about 4 million messages at $N = 2,000$, which made the time to run go up to more than 86 seconds on a single node. The network socket timeouts when N is greater than 10,000, which is a clear example of what not to do on a large scale.

3.2 Group-Based MapReduce (group.py)

The **group-based** method divides elements into G buckets by using element $\% G$. Each unique pair of buckets (g_a, g_b) gets a reducer. The mapper sends every component to all G reducers that are in its bucket. Then, each reducer does either a Cartesian product (cross-bucket pairs) or combinations (same-bucket pairs).

- **Replication rate:** G (each element sent to G reducers)
- **Total network messages:** $N * G$ which makes complexity $O(N * G)$
- **Number of reducers:** $G * \frac{G+1}{2}$

Instead of moving individual rows, we move blocks of data. This cuts **down on network traffic from $O(N^2)$ to $O(N * G)$** . With G set to 10 (the default for the project), **it made this method** much faster across all tested dataset sizes. A bigger G makes the work for each **reducer less but the fan-out more, while** a smaller G does the opposite.

3.3 Spark SQL with Catalyst Optimizer (sql.py)

Instead of manually creating a MapReduce pipeline, this method turns the RDD into a DataFrame, registers it as a temporary view, and runs a self-join SQL query. The Catalyst optimizer and Tungsten execution engine in Spark figure out the physical plan on their own.

```
SELECT /*+ BROADCAST(a) */ a.id AS element_1, b.id AS element_2
FROM   dataset a
JOIN   dataset b ON a.id < b.id
```

The **/*+ BROADCAST */** hint instructs Catalyst to use a Broadcast Hash Join: the entire dataset is broadcast to every executor, avoiding a network shuffle altogether. Without this hint, Catalyst falls back to a Sort-Merge Join, resulting in performance comparable to the naive approach. With the hint, execution time is near-constant across dataset sizes (~ 2.3), making it the fastest of the three strategies.

3.4 Bonus - Afrati-Ullman / Fano Plane (partA_bonus.py)

This implementation is based on the paper "*Matching Bounds for the All-Pairs MapReduce Problem*" (Afrati & Ullman et al.). The paper establishes a **lower bound** on the replication rate (r) necessary to

resolve the All-Pairs problem within a single MapReduce round and formulates optimal designs utilizing combinatorial block designs.

- $N = 7$ elements, 7 reducers.
- Each element assigned to exactly 3 reducers ($r = 3$).
- Every unique pair generated exactly once, which gives zero redundancy.
- Theoretically optimal: no design with $r < 3$ can cover all $C(7,2) = 21$ pairs.

The routing table (*FANO_MAPPING* in *config.py*) maps each element 0-6 to its three assigned reducer IDs. When the script is run on the full dataset ($N > 7$), it clamps the input to the first 7 records to preserve the mathematical guarantees of the design.

4. Performance Evaluation - Part A

All benchmarks were executed on a **local single-node Spark instance** (Windows 10, JDK 11, Spark 3.5.8). The dataset is *person.csv* and *LIMIT_N* controls how many rows are loaded. Execution times are wall-clock seconds measured around the Spark action (count / collect) only (Spark session startup is excluded).

4.1 Results

Table 1: Part A execution times across dataset sizes

N (elements)	Unique Pairs	Naive (s)	Group-Based G=10 (s)	SQL + BROADCAST (s)
500	124,750	4.71	2.05	2.56
1,000	499,500	8.54	1.87	1.56
2,000	1,999,000	85.80	2.52	1.84
5,000	12,497,500	405.19	6.89	1.83

4.2 Analysis

- **Naive MapReduce** scales quadratically: doubling N from 1,000 to 2,000 increases execution time by $\sim 11x$, consistent with $O(N^2)$ shuffle growth.
- **Group-Based MapReduce** scales near-linearly with N for fixed G : the jump from $N=500$ to $N=2,000$ (4x more data) only increases time from 2.05 s to 2.52 s ($\sim 1.2x$), because the number of reducers stays fixed at $G * \frac{G+1}{2} = 55$.
- **SQL + BROADCAST** is effectively constant-time across these sizes (~ 2 s) because the broadcast hash join eliminates the shuffle entirely. It is the fastest approach but relies on the dataset fitting in executor memory.
- At $N = 2,000$, Group-Based achieves a $\sim 32x$ speedup over Naive (2.52 s vs 85.80 s). The gap would widen further at larger N .

5. Part B - Three-Way Join $A(x,y) \Join B(y,z) \Join C(z,w)$

This section looks at how to calculate a **three-way chain join** using three different architectural patterns. All relations (A , B , C) are synthetic and have the same size N . The join attributes come

from a domain of size D . A fixed random seed (42) was used in all implementations to make sure that the results could be repeated.

5.1 Direct Ternary Join (ternary.py)

We implemented a single-round MapReduce strategy in which reducers are logically arranged in a $K * K$ grid ($K=10$ for 100 reducers), and data is routed as follows:

- **A(x,y)** emits to all K reducers in row $i^* = \text{hash}(y) \% K$ (replicated K times).
- **B(y,z)** emits to exactly one reducer at $(\text{hash}(y) \% K, \text{hash}(z) \% K)$ (no replication).
- **C(z,w)** emits to all K reducers in column $j^* = \text{hash}(z) \% K$ (replicated K times).

Because B is routed to a single cell (i^*, j^*) , and A covers the entire row i^* while C covers the entire column j^* , all three matching tuples always co-locate at (i^*, j^*) . The join is completed locally at each reducer. Communication cost: $O(N * K)$. With $K=10$ this uses 100 reducers.

5.2 Two Successive Binary Joins (binary.py)

This approach utilizes a standard two-round hash-join pipeline:

- **Round 1:** Joins A and B on attribute y to produce an intermediate relation $AB(x, y, z)$.
- **Round 2:** Joins the intermediate AB result with C on attribute z .

The extra shuffle between rounds is the main cost. When join selectivity is **high** (small D), the intermediate AB relation can be very large, amplifying the second shuffle significantly.

5.3 Spark SQL - Catalyst Optimizer (join_sql.py)

The join was written as a high-level SQL query, which let Spark's Catalyst Optimizer figure out how to run it. For three relations of about the same size, Catalyst usually picks a **Sort-Merge Join**. This plan avoids full relation replication by hash-partitioning all data through exchange.

```
SELECT A.x, A.y, B.z, C.w
FROM   A
JOIN   B ON A.y = B.y
JOIN   C ON B.z = C.z
```

5.4 Performance Evaluation - Part B

Benchmarks run on a local single-node Spark instance (Windows 10, JDK 11, Spark 3.5.8). Timings are **wall-clock seconds** for the Spark action only (session startup excluded). Two axes were evaluated: selectivity (vary D , fixed $N = 2000$) and scale (vary N , fixed $D = 100$).

Table 2: Part B selectivity benchmark ($N=2000$, D varies)

D (domain)	Result tuples	Ternary (s)	Binary (s)	SQL (s)
50 (high)	3,202,636	79.03	113.59	26.54
200 (medium)	198,870	77.98	113.50	27.00
800 (low)	12,140	77.92	112.49	27.24

Table 3: Part B scale benchmark ($D=100$, N varies)

N (tuples/relation)	Result tuples	Ternary (s)	Binary (s)	SQL (s)
1,000	98,554	78.01	113.56	26.27

5,000	12,545,693	78.80	113.67	26.78
10,000	100,463,224	82.14	149.38	27.59

5.5 Analysis

- **SQL/Catalyst Efficiency:** Spark SQL is still the best option for all configurations, with a consistent execution time of about 26–27 seconds. The Catalyst Optimizer chose **Sort-Merge Join**, which avoids full relation replication.
- **Stability of the Ternary Join:** The direct Ternary Join is the best custom MapReduce implementation, taking between 77 and 82 seconds. Because it finishes in **one round** without creating any intermediate relations, its performance stays very stable even when the selectivity or scale goes up.
- **Binary Join Scalability Issues:** The Binary method is always the slowest, and its latency goes up quickly as the scale gets bigger. At $N = 10,000$, it took **149.38 seconds** to complete because the middle AB relation got so big that it made the second shuffle round much harder.
- **Selectivity Impact:** In this local single-node setup, selectivity (D) had very little effect on execution time, even though it created millions of result tuples. This means that the fixed costs of Spark's shuffle mechanism and RDD management are higher than the costs of the actual join computation at these data volumes.
- **Scale Divergence:** The Ternary Join's benefits are most clear when $N = 10,000$. The Ternary execution time only went up by about 4 seconds (from $N=1,000$ to $N=10,000$), but the Binary join time went up by almost 36 seconds. This shows that the single-round Ternary Join is much more **scalable**.

6. Conclusions

The experiments in **Part A** show very clearly that algorithmic design has a bigger effect on distributed performance than any low-level tuning. The **Naïve** all-pairs strategy doesn't work well at medium scale because it costs too much to communicate, but the **Group-based** strategy cuts down on this cost, making it work almost linearly for fixed G . The **Afrati-Ullman** bonus implementation also shows that information-theoretic limits are very useful, since the **Fano Plane** construction gets a provably optimal replication rate for certain dataset sizes.

In **Part B**, the results show that **single-round joins** are better for performance. The Ternary Join was the most stable custom implementation, keeping execution times stable even when selectivity and scale grew. By finishing the join in one round, it avoids the cost of making intermediate relations. **Binary Joins**, on the other hand, were always the slowest and least scalable because the system had to deal with a lot of overhead from multiple shuffles and the growth of intermediate data at larger scales.

In terms of raw performance, **Spark SQL** with the **Catalyst optimizer** was the clear winner in both parts of the project. SQL used **Sort-Merge Joins** to cut down on the costs of Python serialization and make physical plans work better than manual RDD-based implementations. The high-level SQL layer is better for production, but the main benefit of MapReduce implementations is that they can show

the basic trade-offs between the cost of communication, the load on the computer, and the characteristics of the data in a distributed environment.

7. Execution Instructions

Prerequisites: Java JDK 11.0.0.2, Hadoop winutils (Windows), Apache Spark 3.5.8, Python 3.11.0 with packages: pyspark, findspark.

Run Part A scripts from the project root directory:

```
python src/partA/naive.py
python src/partA/group.py
python src/partA/sql.py
python src/partA/partA_bonus.py
```

Run Part B scripts individually:

```
python src/partB/ternary.py
python src/partB/binary.py
python src/partB/join_sql.py
```

To run the full benchmark matrix for both Part A and Part B in a single Spark session:

```
python src/benchmark.py
```

The unified benchmark runner reads its test matrices directly from config.py (BENCH_A_SIZES, BENCH_B_SELECTIVITY, BENCH_B_SCALE) and writes results to benchmark_output.txt in the project root. To add or remove test sizes, edit only the relevant list in config.py, no changes to benchmark.py are required.

To switch between test and real-data mode, edit config.py:

```
TEST_MODE = True    # uses MOCK_N=100 synthetic integers (fast, for debugging)
TEST_MODE = False   # loads person.csv and limits to LIMIT_N rows (default: 500)
```

The individual Part A scripts (naive.py, group.py, sql.py, partA_bonus.py) and Part B scripts (ternary.py, binary.py, join_sql.py) each use config.LIMIT_N and config.PART_B_N / config.PART_B_D respectively. The benchmark runner uses its own config.BENCH_* matrices, which are independent of LIMIT_N.